Overview

The objective of the mini-project is to build a small, self-contained computational project that uses numerical methods (ideally from this course) to solve a problem relevant to engineering/astronautics and your own interests. The emphasis is on **process**, **effort**, and **understanding**, not a perfectly polished final project.

Your mini-project will be **due 30 minutes before your in-person code review**. This will give me time to clone your repo so we can discuss it together. Since the code reviews are held throughout finals week, this means that everyone will have different "due dates." I will take this into consideration for those who schedule code reviews earlier in the week, since they will have less time overall to complete the mini-project.

Suggested Project Types

a) **Extension of existing class work**

- Extend a homework or example code (e.g. flesh out the rocket-relations repo, test new time integrators, build an interactive visualization, add new solvers and convergence studies, etc.)

b) **Self-contained original program/app/package**

- A new codebase with a clearly-defined numerical goal (e.g. ODE/PDE solver, Monte Carlo, optimization, root-finding, interpolation, data assimilation, simplified toy model, interactive web app, trade tool, etc.)

Minimum Project Requirements

Your project must include **all** of the following:

1. **Numerical core**
   - At least one "real" numerical method (e.g. finite differences, iterative linear solvers, time integration, root finding, interpolation, Monte Carlo estimation, optimization, etc.)
     - The method need not be implemented from scratch (i.e. you can use external libraries)
   - You must explain *why* the chosen method is appropriate and what its limitations/stability constraints are
2. **Pathway to verification**
   - Correctness of the code and implementation is not strictly required
   - However, you must demonstrate that you know how you *would* establish correctness
     - E.g. an analytic comparison on a simplified test case, grid refinement/convergence trends, conservation checks, unit tests, accounting for edge cases, etc.
3. **Version control with GitHub**
   - Work must be tracked in a GitHub repo with meaningful commits over time (not one giant upload at the end)*
   - The repo must include a `README.md` with build/run instructions. A fresh user must be able to run this on their machine by following the `README.md`
4. **Main deliverables**
   (a) **PDF report** following requirements listed below
   (b) **Source code** provided via GitHub link to repo (with access granted to me if private)

Report requirements

The report should look similar to other homework assignments. Remember: it's not a formal paper, but rather a transparent record of the development and engineering process.

Where relevant, you should include:

1. **Project overview:** What you built, why it matters, what question you're answering
2. **Numerical method(s):** Equations, discretization/algorithm (pseudocode is fine), stability/accuracy considerations/limitations
3. **Implementation notes:** Key design choices, file/directory layout, how to run (similar to README)
4. **Progress log:** Dated entries or milestones (what you tried, what broke, what you changed, what you learned)
5. **Verification/validation evidence:** Even just partial is okay here. Show what you checked, what you were thinking about, and what remains
6. **Results:** Plots/tables/screenshots—interpret what's shown (and feel free to ask me for help if necessary)
7. **Reflection/next steps:** What would you do if you had more time? What did you learn?
8. **LLM transcripts:** Include it all at the end, each individual conversation sorted by date. Feel free to reference this in the progress log or elsewhere

Note for those who started work before requirements were posted

Some of you may have started working on this mini-project before these instructions were uploaded on Brightspace. That's okay. To put you at ease, I'm grandfathering in such work:

- *You will not be penalized* if your early work doesn't exactly match the structure/repo/report expectations above, though try your best to fit it within that structure
- To receive full credit, your final submission must make it possible for me to *see and evaluate the work you did*—even if it occurred earlier—either via:
  - Git commit history (preferred),
  - a clearly labeled "pre-requirements work" section in the report with dated screenshots/notes, or
  - both of these

If you already wrote lots of code without Git, *you can still get full credit* by initializing a repo now (no later than 12/4 EOD after I mention this in class), committing the current state as "Initial import (work completed pre-rubric posting)" and then continuing with incremental commits.

Points and Grading (20 pts total)

This mini-project is worth **20 points** in your homework grade and *cannot be dropped*. Since only your top 6 homework assignment grades will be used for the overall class grade, this equates to 25% of your overall homework grade and thus 15% of your semester grade. The in-person code review is evaluated separately from the mini-project itself.

As mentioned in class, the expectation is that the mini-project represents approximately 3 weeks worth of work. As I say at the beginning of this document, I will grade primarily on your engineering process, overall effort, and demonstrated understanding—not on whether your final code is "perfect." I care most about you

putting in careful thought and intentionality in your efforts, and it should be easy to find evidence of this in your overall body of work here.

The rubric I will be using to evaluate your work is below. I reserve the right to assign half points, as well as zero points for missing parts, and in truly exceptional cases I may consider awarding an extra credit point. If you have an idea for the mini-project that doesn't fit this rubric exactly (i.e. it relies much more heavily on visualization or software engineering, e.g. a web app) please let me know ASAP and we can agree on a slight adjustment.

Rubric

Category (4 pts each)	**1** Minimal	**2** Developing	**3** Solid	**4** Thorough
Overall Effort & Initiative	Little evidence of sustained work; few iterations; limited progress.	Some effort, but uneven progress; limited iteration or depth.	Clear progress and effort; at least one meaningful refinement pass.	Strong sustained effort; multiple iterations; obstacles addressed; exceeds minimum in at least one way.
Numerical Methods & Understanding	Little evidence of understanding of the numerical method(s) used.	Uses a numerical method but explanation is incomplete or partially confused; weak justification.	Correct description and reasonable justification; mentions at least one limitation or stability/accuracy issue.	Well-justified method choice; explains assumptions, stability/accuracy expectations, and failure modes; connects numerics to observed behavior.
Verification & Testing	No real verification plan or evidence.	Only informal checks, or verification plan is vague.	At least one meaningful verification attempt plus a credible plan for additional checks.	Multiple verification checks (analytic solution comparison, grid refinement/convergence trends, conservation checks, unit/regression tests) with interpretation.
Software Engineering Quality	Hard to run or evaluate; minimal GitHub usage or reproducibility.	Repo exists but run steps are unclear/fragile; sparse commits; messy organization.	Repo is usable; README mostly sufficient; commits show progress; minor rough edges.	Clear structure; README enables a fresh user to run; deps/env specified; meaningful commits; scripted runs and/or basic tests.
Report/ Engineering Log	Minimal report; hard to determine what was done.	Report exists but thin/disorganized; missing key evidence or interpretation.	Good report with progress evidence; objective mostly clear; some reflection.	Clear objective/scope; structured progress log; plots/screenshots; key decisions; failures/pitfalls; concrete next steps.

Other Miscellaneous Thoughts

Submit code that runs

Your final product *should* be something that actually runs. If a portion doesn't work properly, comment it out and replace it with a minimal example. If you wish to demonstrate something that isn't working and needs further development, toss it into a separate branch using Git and we can discuss it in your code review. If you need help with this, schedule a time to discuss with me in office hours or during study days.

"Vibe Coding" with LLMs encouraged

You are *encouraged* (but not required) to use an LLM for at least one subtask (e.g. plotting routine, CLI wrapper, unit test skeleton, refactor, documentation, data parsing, etc.) up to and including the entire mini-project code. This is entirely optional; you can abandon it if it doesn't help.

I find that brainstorming initial ideas or project structures with a chat-based LLM like ChatGPT can be helpful at the beginning of a project to supplement my own notes and think about software engineering concerns from the outset. With an overall framework and high-level plan in mind, you can use GitHub Copilot for "vibe coding" a first draft of the code from start-to-finish. The more detailed your prompt, the better your outcome. Similarly, the larger the ask (i.e. more code output), the worse the outcome. Context is everything. To me, "vibe coding" means writing an entire program/file's worth of code from scratch and using only LLM prompts to iterate. Personally, I find diminishing returns with this approach unless I know exactly what I want and already have a strong idea of how I'd get there, in which case it's usually just as fast/efficient for me to do it myself.

If "vibe coding" produces a substantial portion of your code, you should **include a short "LLM dev log"** section in your report, detailing:

- What you asked for (reference your transcript and include snippets or a summary with key prompts)
- What worked quickly
- What went wrong: where it got stuck, any hallucinations, debugging you had to do
- What you changed manually afterward
- A brief takeaway: "When would I use this approach again or not?"

Note that the use of vibe coding as described here *cannot hurt your grade*. If included, it can strengthen your report and various other sections (most likely the software engineering quality) by demonstrating a modern workflow and good judgment. However, if you don't use an LLM, you can just as easily still earn full credit.

Additional guidelines:

- **Parameterization:** Avoid hard-coding everything. We didn't have time to cover substantial examples, but a great program/app will read an input/config file or accept command-line interface (CLI) arguments where reasonable.
- **Diagnosis plot:** In addition to a plot for results (which would indicate understanding of the science/engineering problem), be sure to include *one plot that actually diagnoses something* to satisfy the verification requirement. This could be something like the plotting the residual vs. iteration, error vs grid spacing, stability behavior vs. time step, etc.